

1. A university wants to determine whether students who spend more hours studying obtain higher marks.

```
StudyHours <- c(2, 3, 4, 5, 6, 7, 8, 9)
```

```
Marks <- c(45, 52, 58, 63, 69, 74, 81, 88)
```

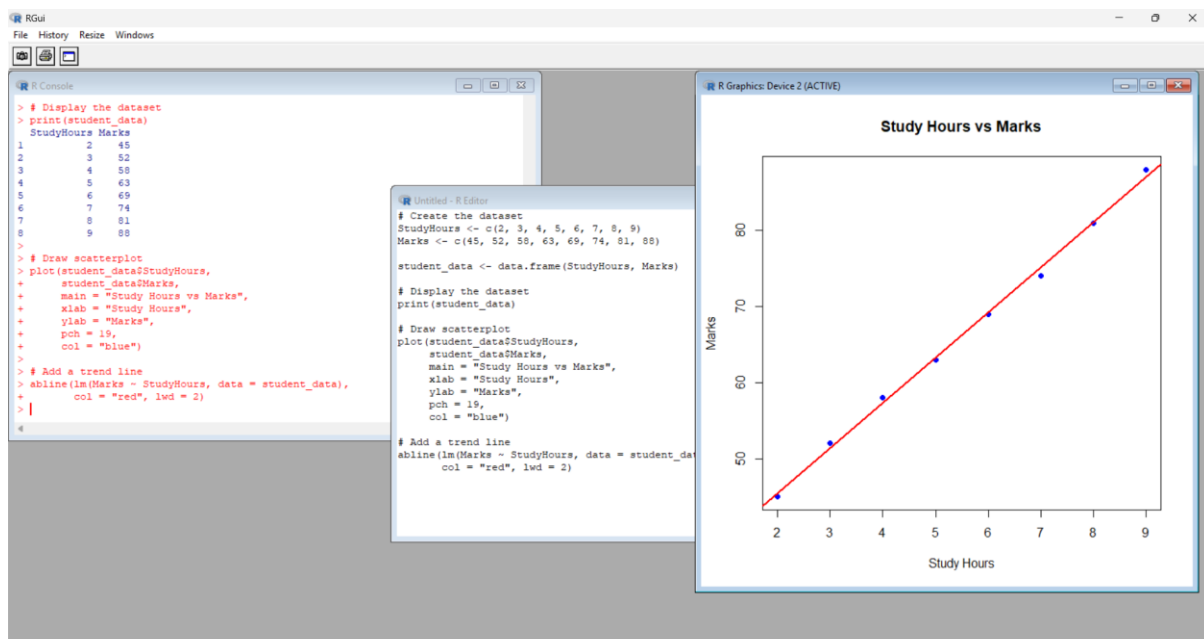
```
student_data <- data.frame(StudyHours, Marks)
```

```
print(student_data)
```

```
plot(student_data$StudyHours,  
      student_data$Marks,  
      main = "Study Hours vs Marks",  
      xlab = "Study Hours",  
      ylab = "Marks",  
      pch = 19,  
      col = "blue")
```

```
# Add a trend line
```

```
abline(lm(Marks ~ StudyHours, data = student_data),  
       col = "red", lwd = 2)
```



2. A hospital records patients' age and systolic blood pressure.

Python Code:

```
import matplotlib.pyplot as plt
```

```
age = [25, 30, 35, 40, 45, 50, 55, 60]
```

```
bp = [110, 115, 118, 122, 128, 135, 140, 148]
```

```
plt.scatter(age, bp, color='red')
```

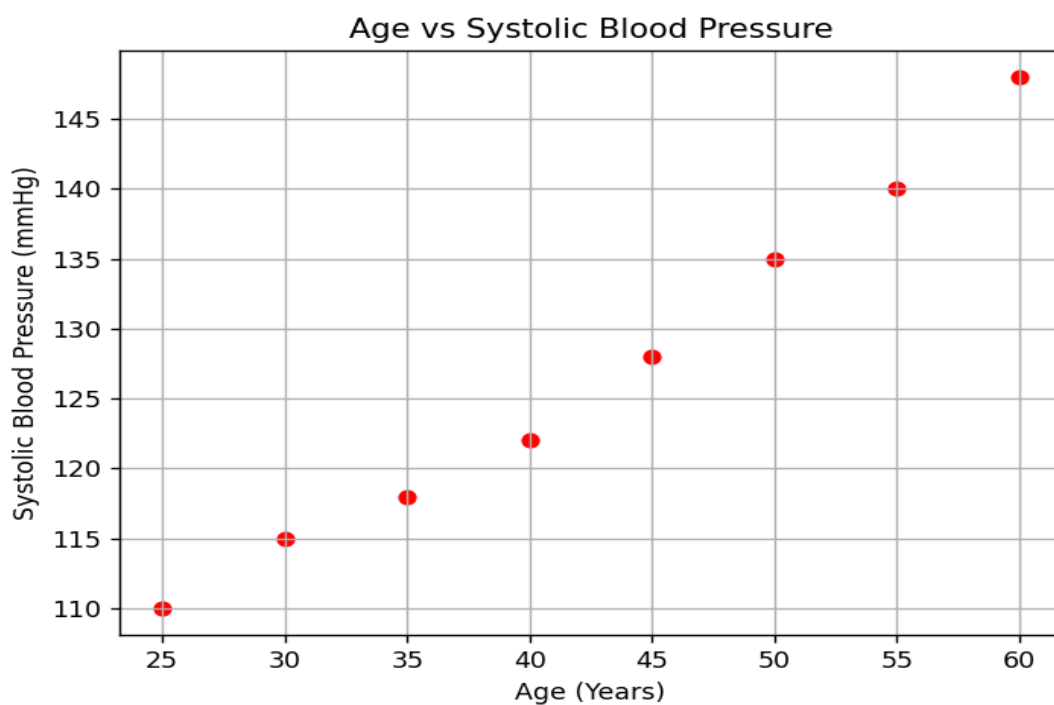
```
plt.title("Age vs Systolic Blood Pressure")
```

```
plt.xlabel("Age (Years)")
```

```
plt.ylabel("Systolic Blood Pressure (mmHg)")
```

```
plt.grid(True)
```

```
plt.show()
```



3. A company wants to investigate whether advertising expenditure influences monthly sales. Develop a scatterplot and determine whether sales increase with advertising expenditure.

```
import matplotlib.pyplot as plt
```

```
advertising = [10,20,30,40,50,60,70,80]
```

```
sales = [100,120,135,150,170,190,210,230]
```

```
plt.scatter(advertising, sales, color='blue')
```

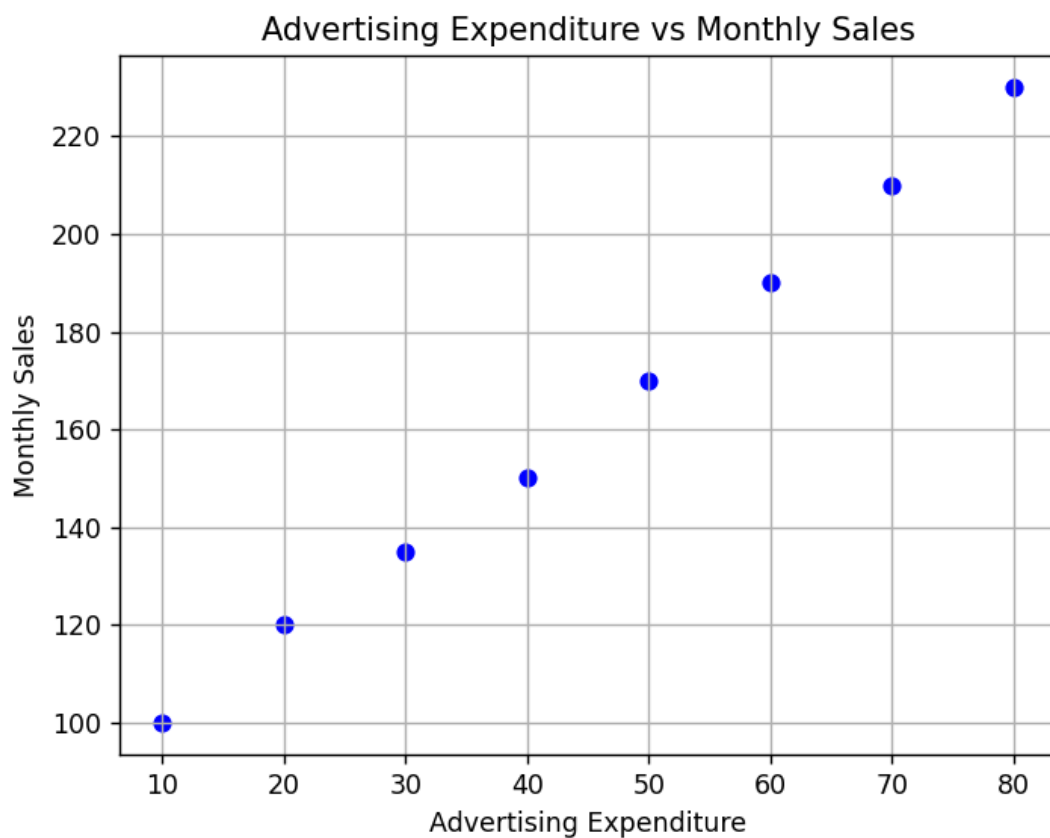
```
plt.title("Advertising Expenditure vs Monthly Sales")
```

```
plt.xlabel("Advertising Expenditure")
```

```
plt.ylabel("Monthly Sales")
```

```
plt.grid(True)
```

```
plt.show()
```



4. A financial institution has collected information on:

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
data = {  
    'Income': [30000, 35000, 40000, 45000, 50000, 55000],  
    'Savings': [5000, 7000, 9000, 11000, 13000, 15000],  
    'Loan Amount': [10000, 12000, 15000, 18000, 21000, 25000],  
    'Credit Score': [620, 650, 680, 700, 730, 760]  
}
```

```
df = pd.DataFrame(data)
```

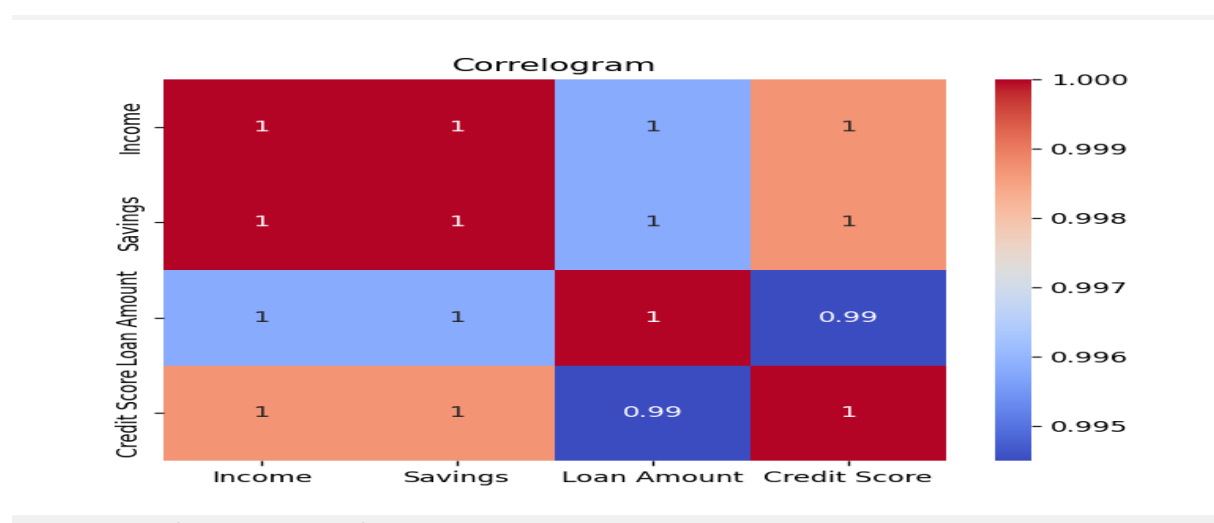
```
corr = df.corr()
```

```
print(corr)
```

```
sns.heatmap(corr, annot=True, cmap='coolwarm')
```

```
plt.title("Correlogram")
```

```
plt.show()
```



5. Using the Iris dataset,

Compute the correlation matrix.

Display it as a heatmap using Seaborn.

Identify variables having negative correlation.

```
from sklearn.datasets import load_iris
```

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)
```

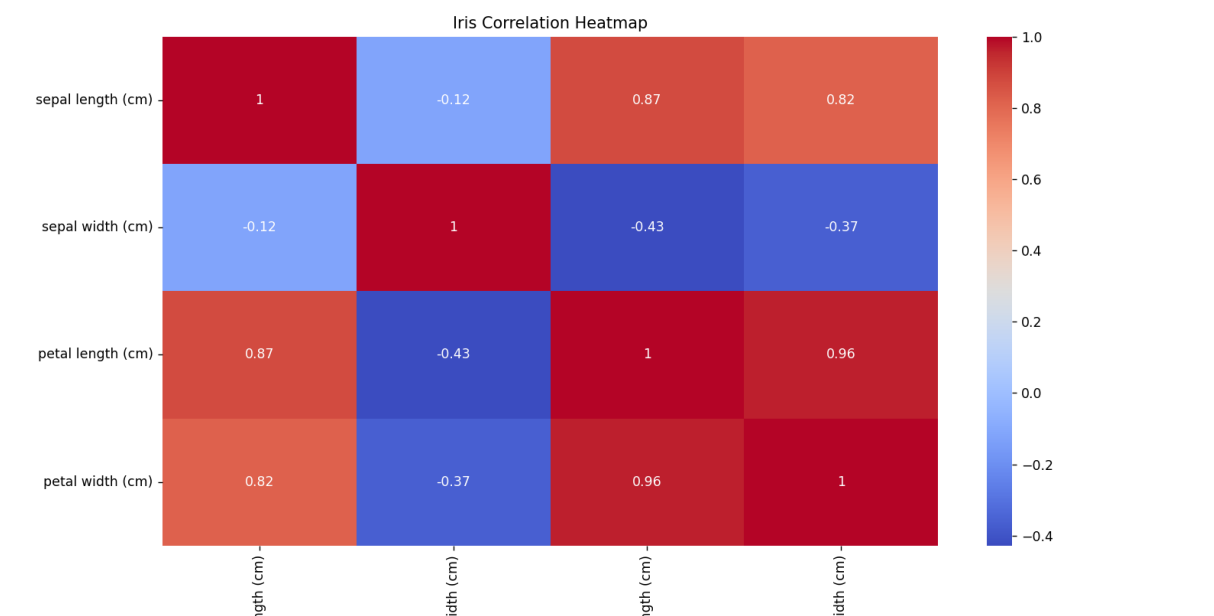
```
corr = df.corr()
```

```
print(corr)
```

```
sns.heatmap(corr, annot=True, cmap='coolwarm')
```

```
plt.title("Iris Correlation Heatmap")
```

```
plt.show()
```



6. A healthcare dataset

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

data = {

'BMI':[22,24,26,28,30,32],

'Cholesterol':[180,190,200,210,220,230],

'Blood Pressure':[110,115,120,130,135,140],

'Glucose':[90,95,100,110,120,130],

'Heart Rate':[70,72,74,76,78,80]

}

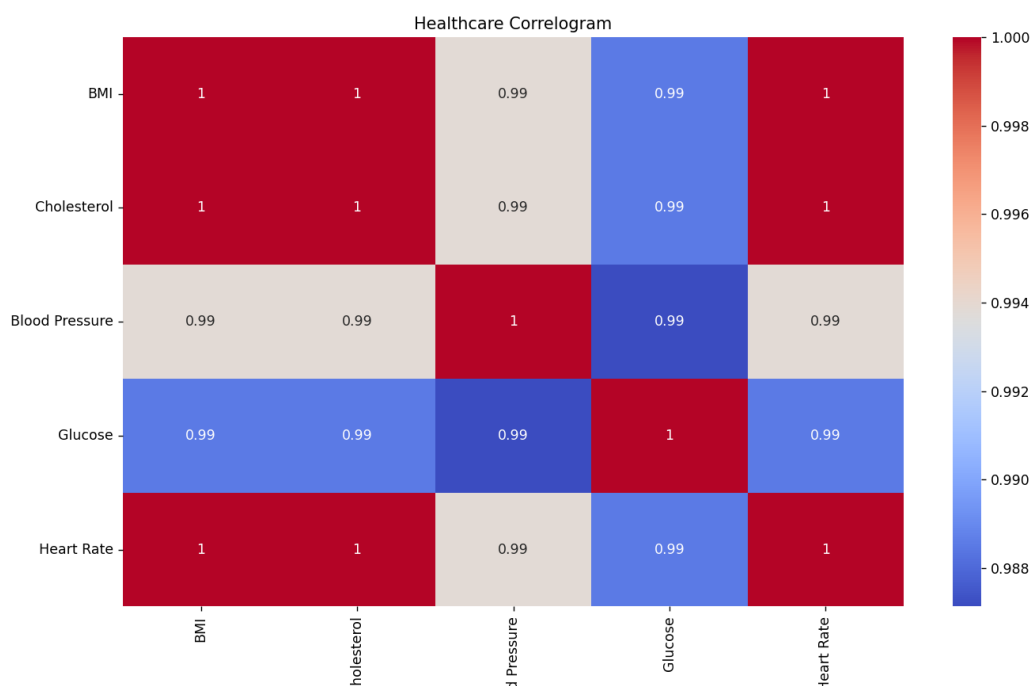
df = pd.DataFrame(data)

corr = df.corr()

sns.heatmap(corr, annot=True, cmap='coolwarm')

plt.title("Healthcare Correlogram")

plt.show()
```



7. A researcher is analyzing 25 features extracted from MRI images. Perform PCA to reduce the dimensionality to two principal components. Plot the transformed data.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.decomposition import PCA
```

```
X = np.random.rand(100,25)
```

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X)
```

```
plt.scatter(X_pca[:,0],X_pca[:,1])
```

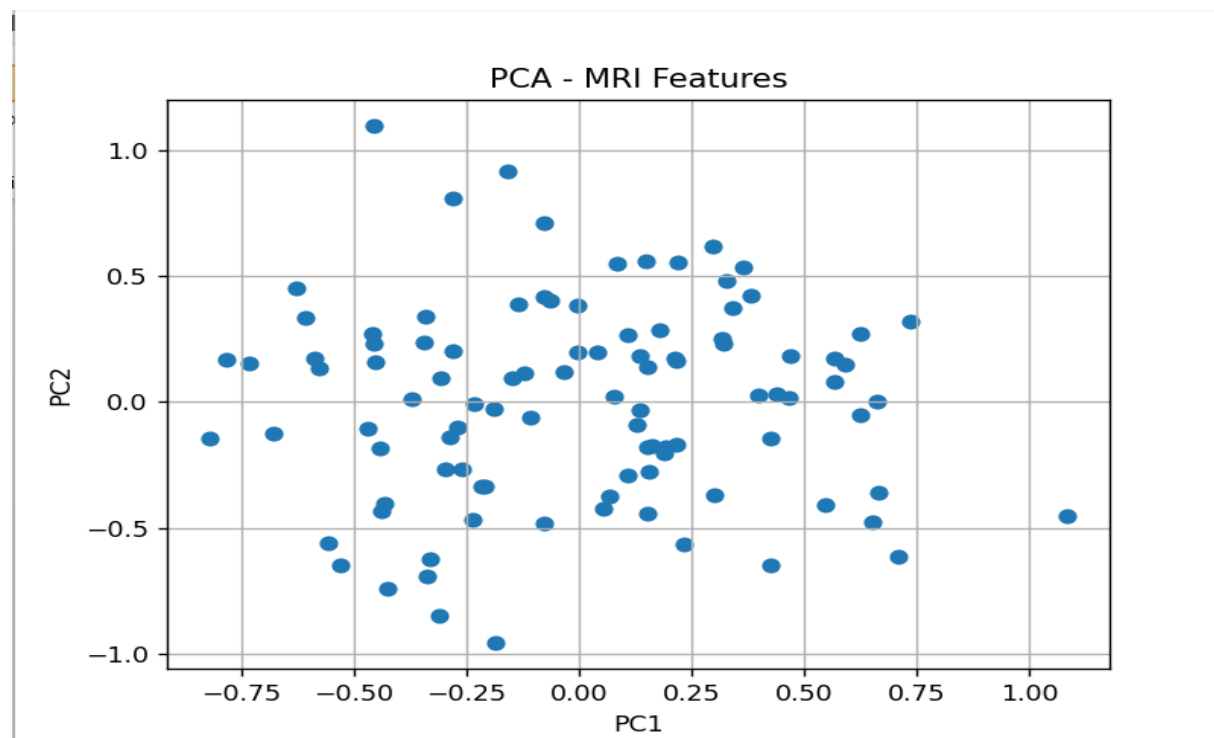
```
plt.title("PCA - MRI Features")
```

```
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
```

```
plt.grid(True)
```

```
plt.show()
```



8. Using the Iris dataset

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

```
X = iris.data
```

```
pca = PCA()
```

```
X_pca = pca.fit_transform(X)
```

```
print("Explained Variance Ratio")
```

```
print(pca.explained_variance_ratio_)
```

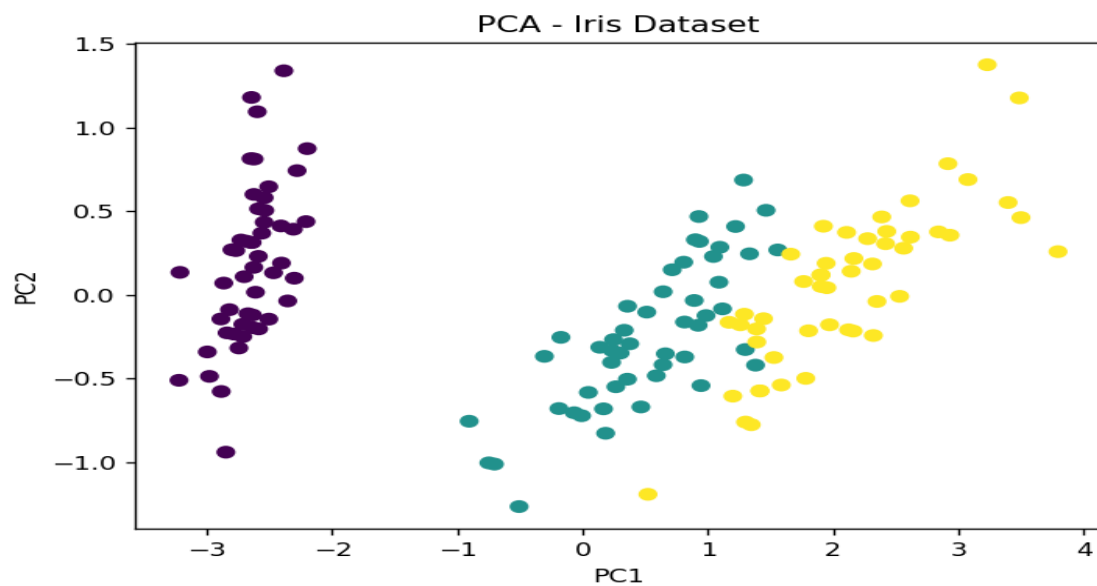
```
plt.scatter(X_pca[:,0],X_pca[:,1],c=iris.target)
```

```
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
```

```
plt.title("PCA - Iris Dataset")
```

```
plt.show()
```



9. An automobile company has recorded 15 specifications for various cars. Reduce the dimensions using PCA and visualize the first two principal components.

```
import numpy as np

from sklearn.decomposition import PCA

import matplotlib.pyplot as plt

X = np.random.rand(100,15)

pca = PCA(n_components=2)

result = pca.fit_transform(X)

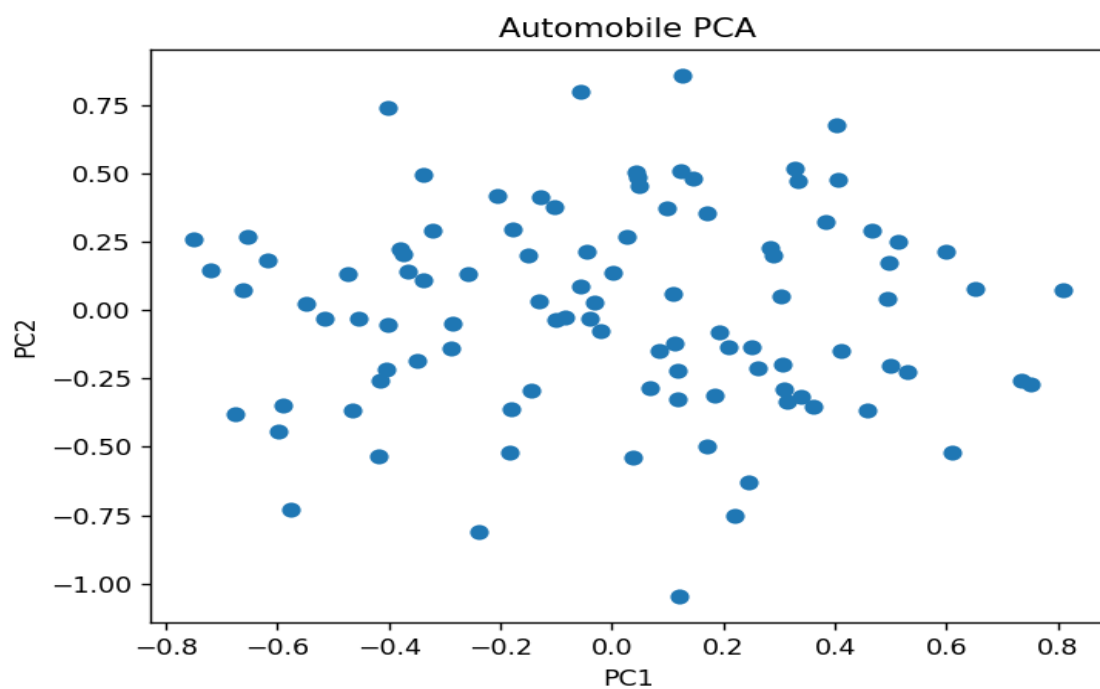
plt.scatter(result[:,0],result[:,1])

plt.title("Automobile PCA")

plt.xlabel("PC1")

plt.ylabel("PC2")

plt.show()
```



10. A facial recognition dataset contains 128-dimensional facial embeddings.

```
import numpy as np

from sklearn.manifold import TSNE

import matplotlib.pyplot as plt

X = np.random.rand(200,128)

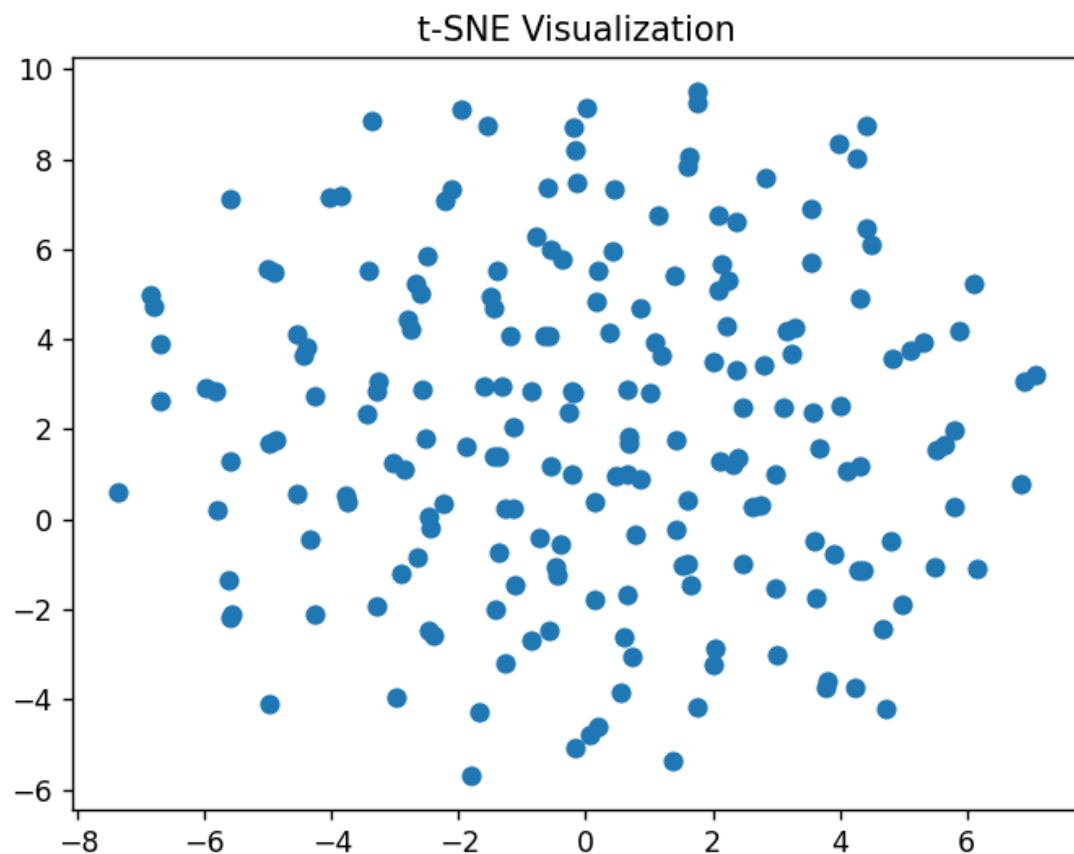
tsne = TSNE(n_components=2, random_state=42)

X_tsne = tsne.fit_transform(X)

plt.scatter(X_tsne[:,0],X_tsne[:,1])

plt.title("t-SNE Visualization")

plt.show()
```



11. Using the Iris dataset,

Apply t-SNE and plot the resulting clusters using different colors for each species.

```
from sklearn.datasets import load_iris
```

```
from sklearn.manifold import TSNE
```

```
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

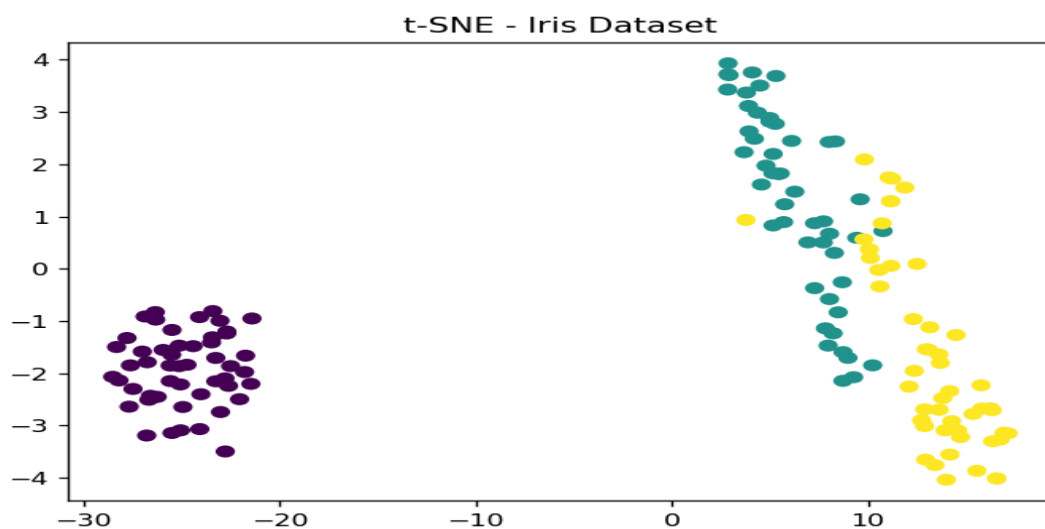
```
tsne = TSNE(n_components=2, random_state=42)
```

```
X_tsne = tsne.fit_transform(X)
```

```
plt.scatter(X_tsne[:,0],X_tsne[:,1],c=y)
```

```
plt.title("t-SNE - Iris Dataset")
```

```
plt.show()
```



12. A customer segmentation dataset contains 40 behavioral variables.

```
import numpy as np

from sklearn.manifold import TSNE

import matplotlib.pyplot as plt

X = np.random.rand(150,40)

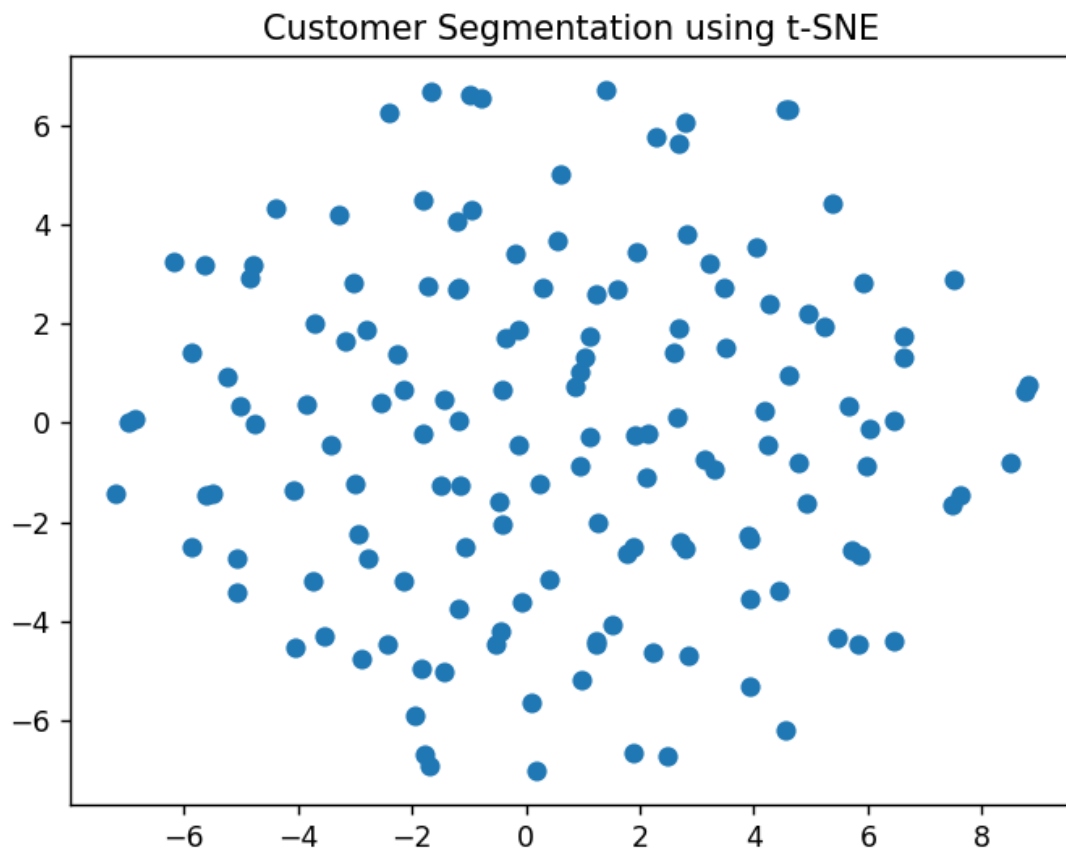
tsne = TSNE(n_components=2, random_state=42)

result = tsne.fit_transform(X)

plt.scatter(result[:,0],result[:,1])

plt.title("Customer Segmentation using t-SNE")

plt.show()
```



13. A medical image dataset contains 100 extracted features.

```
import numpy as np

import matplotlib.pyplot as plt

from umap import UMAP

X = np.random.rand(200, 100)

reducer = UMAP(n_components=2, random_state=42)

embedding = reducer.fit_transform(X)

plt.figure(figsize=(6,5))

plt.scatter(embedding[:, 0], embedding[:, 1], s=30)

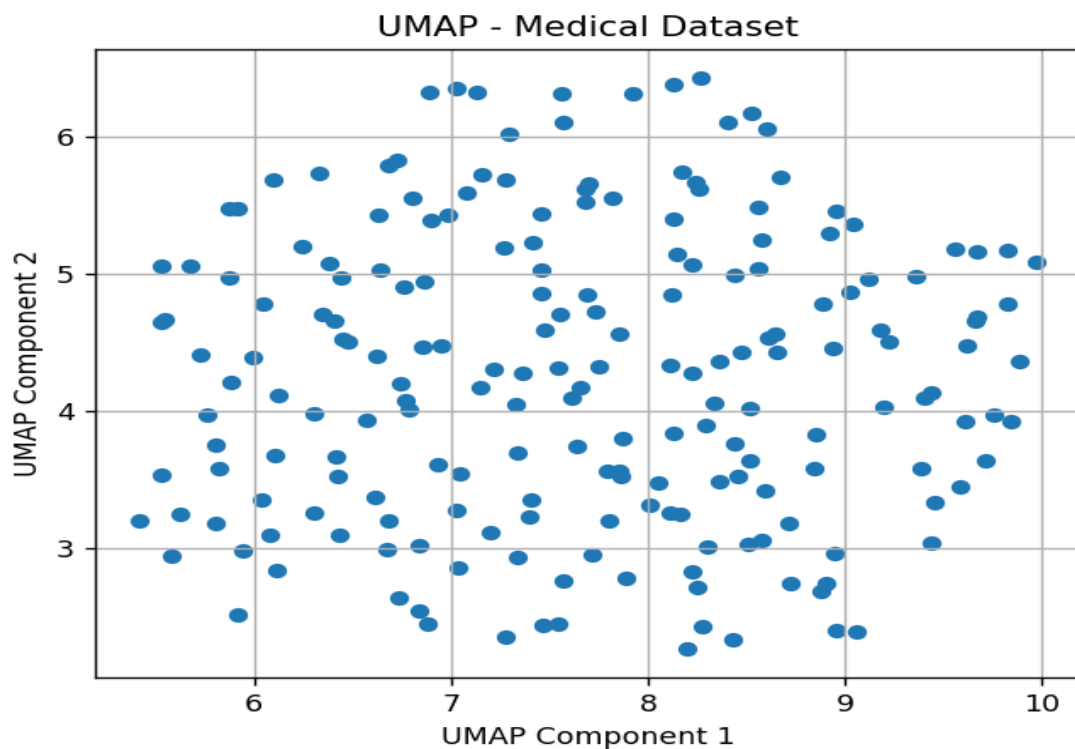
plt.title("UMAP - Medical Dataset")

plt.xlabel("UMAP Component 1")

plt.ylabel("UMAP Component 2")

plt.grid(True)

plt.show()
```



14. Apply UMAP to the Iris dataset.

Plot the reduced dimensions using different colors for each species.

```
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
import umap

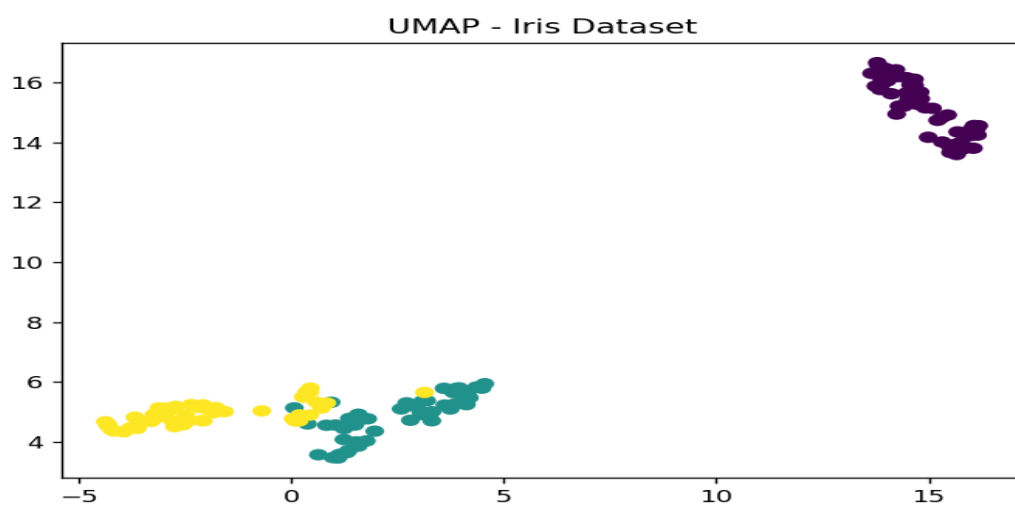
iris = load_iris()

X = iris.data
y = iris.target

reducer = umap.UMAP(random_state=42)

embedding = reducer.fit_transform(X)

plt.scatter(embedding[:,0],embedding[:,1],c=y)
plt.title("UMAP - Iris Dataset")
plt.show()
```



15. An e-commerce company has collected customer purchasing behavior consisting of 60 attributes.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import umap
```

```
X = np.random.rand(200,60)
```

```
reducer = umap.UMAP(random_state=42)
```

```
embedding = reducer.fit_transform(X)
```

```
plt.scatter(embedding[:,0],embedding[:,1])
```

```
plt.title("Customer Purchasing Behavior - UMAP")
```

```
plt.show()
```

